

2. Git 的常用分支介绍

2.1 Production 分支

也就是我们经常使用的 Master 分支，这个分支最近发布到生产环境的代码，最近发布的 Release，这个分支只能从其他分支合并，不能在这个分支直接修改。

2.2 Develop 分支

这个分支是我们的主开发分支，包含所有要发布到下一个 Release 的代码，这个主要合并与其他分支，比如 Feature 分支。

2.3 Feature 分支

这个分支主要是用来开发一个新的功能，一旦开发完成，我们合并回 Develop 分支进入下一个 Release。

2.4 Release分支

当你需要一个发布一个新 Release 的时候，我们基于 Develop 分支创建一个 Release 分支，完成 Release 后，我们合并到 Master 和 Develop 分支。

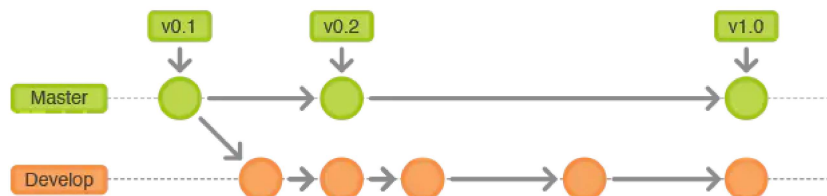
2.5 Hotfix分支

当我们在 Production 发现新的 Bug 时候，我们需要创建一个 Hotfix，完成 Hotfix 后，我们合并回 Master 和 Develop 分支，所以 Hotfix 的改动会进入下一个 Release。

3. Git Flow 各分支操作原理示意

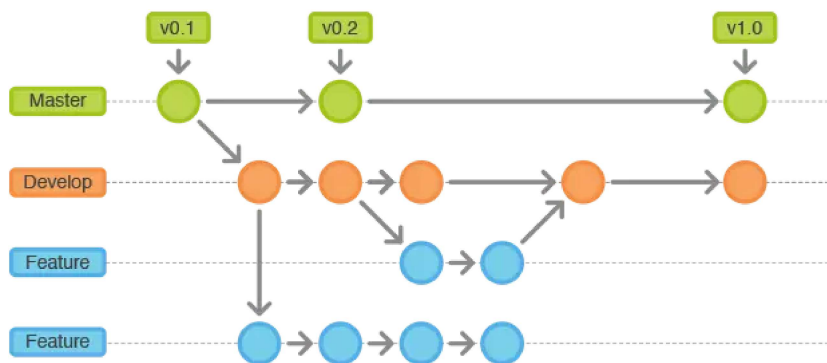
3.1 Master/Develop 分支

所有在 Master 分支上的 Commit 应该打上 Tag，一般情况下 Master 不存在 Commit，Develop 分支基于 Master 分支创建。



3.2 Feature 分支

Feature 分支做完后，必须合并回 Develop 分支，合并完分支后一般会删掉这个 Feature 分支，毕竟保留下来意义也不大。



3.3 Release 分支

2.3 Feature 分支

2.4 Release分支

2.5 Hotfix分支

3. Git Flow 各分支操作原理示意

3.1 Master/Develop 分支

3.2 Feature 分支

3.3 Release 分支

3.4 Hotfix 分支

4. Git Flow 命令示例

4.1 Develop相关

4.2 Feature相关

4.3 Release相关

4.4 Hotfix相关

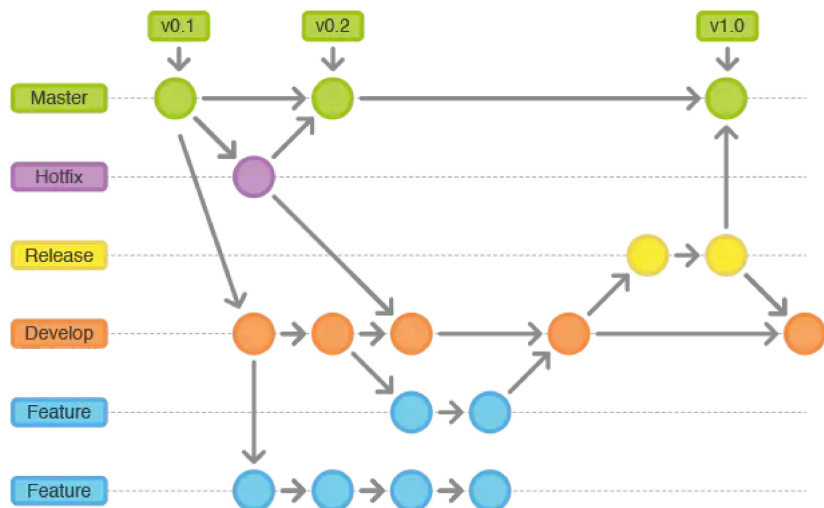
联系作者：QQ群【883210148】，关注 ▼

Release 分支基于 Develop 分支创建，打完 Release 分支之后，我们可以在这个 Release 分支上测试，修改 Bug 等。同时，其它开发人员可以基于 Develop 分支新建 Feature (记住：一旦打了 Release 分支之后不要从 Develop 分支上合并新的改动到 Release 分支)发布 Release 分支时，合并 Release 到 Master 和 Develop，同时在 Master 分支上打个 Tag 记住 Release 版本号，然后可以删除 Release 分支了。



3.4 Hotfix 分支

hotfix 分支基于 Master 分支创建，开发完后需要合并回 Master 和 Develop 分支，同时在 Master 上打一个 tag。



4. Git Flow 命令示例

4.1 Develop 相关

- 创建 develop
git branch develop
git push -u origin develop

4.2 Feature 相关

- 开始 Feature
通过 develop 新建 feature 分支
git checkout -b feature develop
或者, 推送至远程服务器:
git push -u origin feature
修改 md 文件
git status
git add .
git commit

2.3 Feature 分支

2.4 Release 分支

2.5 Hotfix 分支

3. Git Flow 各分支操作原理示意

3.1 Master/Develop 分支

3.2 Feature 分支

3.3 Release 分支

3.4 Hotfix 分支

4. Git Flow 命令示例

4.1 Develop 相关

4.2 Feature 相关

4.3 Release 相关

4.4 Hotfix 相关

联系作者: QQ群【883210148】， 关注 ▼

- 完成 Feature
git pull origin develop
git checkout develop
--no-ff: 不使用 fast-forward 方式合并, 保留分支的 commit 历史
--squash: 使用 squash 方式合并, 把多次分支 commit 历史压缩为一次
git merge --no-ff feature
git push origin develop
git branch -d some-feature
如果需要删除远程 feature 分支:
git push origin --delete feature

4.3 Release相关

- 开始 Feature
git checkout -b release-0.1.0 develop
- 完成 Feature
git checkout master
git merge --no-ff release-0.1.0
git push
git checkout develop
git merge --no-ff release-0.1.0
git push
git branch -d release-0.1.0
git push origin --delete release-0.1.0
合并 master/develop 分支之后, 打上 tag
git tag -a v0.1.0 master
git push --tags

4.4 Hotfix相关

- 开始 Hotfix
git checkout -b hotfix-0.1.1 master
- 完成 Hotfix
git checkout master
git merge --no-ff hotfix-0.1.1
git push
git checkout develop
git merge --no-ff hotfix-0.1.1
git push
git branch -d hotfix-0.1.1
git push origin --delete hotfix-0.1.1
git tag -a v0.1.1 master
git push --tags

联系作者: QQ群【883210148】, 关注公众号【互联网专栏】获取更多资源



微信搜一搜



互联网专栏

2.3 Feature 分支

2.4 Release分支

2.5 Hotfix分支

3. Git Flow 各分支操作原理示意

3.1 Master/Develop 分支

3.2 Feature 分支

3.3 Release 分支

3.4 Hotfix 分支

4. Git Flow 命令示例

4.1 Develop相关

4.2 Feature相关

4.3 Release相关

4.4 Hotfix相关

联系作者: QQ群【883210148】, 关注